

DAY 4

LangChain for Insurance Knowledge Assistants

FastAPI + Swagger + LangChain + Structured Responses + Insurance Assistants

4 Hours | Beginner to Intermediate | UiPath Developers | Insurance Automation

Program Outcome

Participants will build an Insurance Policy Assistant using LangChain and expose insurance AI assistance capabilities through FastAPI endpoints that can later be consumed by UiPath workflows.

Target Audience

- UiPath Developers and RPA Developers transitioning into AI Automation Engineering.
- Automation leads who need to expose AI functions through APIs.
- Insurance technology teams working with claims, policy servicing, customer support and knowledge assistants.

Day 4 Learning Objectives

- Understand FastAPI and Swagger UI using insurance claim assistance examples.
- Understand LangChain and why it is useful for building knowledge assistants.
- Create prompt templates and reusable chains for insurance workflows.
- Use output parsers to generate structured responses for UiPath consumption.
- Build policy FAQ, customer support, claim status and policy summary assistants.
- Design simple tools and agent-style workflows for insurance operations.

Time	Module	Hands-on Outcome
30 min	FastAPI and Swagger UI quick introduction	Run /claim-assist endpoint and test from Swagger UI
40 min	LangChain overview and setup	Create LLM wrapper and first insurance prompt
45 min	Prompt templates and chains	Build policy FAQ assistant
35 min	Output parsers and structured responses	Return JSON-like structured policy answers
35 min	Customer support and claim status assistants	Create reusable support chains
35 min	Tools and simple agents	Simulate policy lookup and claim status tools
40 min	Reusable policy summary chain	Generate policy summary for operations
20 min	Final assignment briefing	Build Insurance Policy Assistant

Part A: Environment and Setup

Assumptions

- API key is already provided to all participants.
- Environment variable is already configured, for example `OPENROUTER_API_KEY`.
- Participants use Python 3.11+, VS Code or Jupyter Notebook, and terminal/command prompt.
- The examples use OpenAI-compatible API access with OpenRouter-style base URL.

Required Libraries

```
pip install fastapi uvicorn python-dotenv openai langchain
langchain-core langchain-openai pydantic requests
```

Trainer Note

For beginner participants, install libraries before the class or provide a ready-to-use

requirements.txt file. Keep the first 15 minutes focused on validating environment setup, not debugging every machine individually.

Part B: FastAPI and Swagger UI

1. What is FastAPI?

- FastAPI is a Python framework used to create APIs quickly.
- An API allows UiPath, web apps or other systems to call Python AI logic.
- Swagger UI is auto-generated API documentation and testing interface.
- For Day 4, FastAPI is used to expose insurance AI assistance as an endpoint.

Simple Analogy

FastAPI is like a service counter. UiPath submits a claim note. Python processes it using the LLM. FastAPI returns a clean response. Swagger UI is the counter demo screen where we test the service before connecting UiPath.

2. FastAPI Claim Assistance Example: Step-by-Step

1. Import required libraries: FastAPI creates the API, Pydantic validates request data, dotenv loads environment variables, and OpenAI calls the LLM.
2. Load environment variables so the API key and model settings are not hard-coded.
3. Create the OpenAI-compatible client using `base_url` and `api_key`.
4. Create a FastAPI app with a title. This title appears in Swagger UI.
5. Define a request model named `ClaimRequest`. It ensures that every API request contains `claim_note`.
6. Create `call_llm()` as a reusable helper function.
7. Create `/claim-assist` endpoint using `@app.post()`.
8. Build an insurance-specific prompt inside the endpoint.
9. Return status and `ai_response` as JSON.

```

from fastapi import FastAPI
from pydantic import BaseModel
import os
from dotenv import load_dotenv
from openai import OpenAI

load_dotenv()
OPENROUTER_API_KEY = os.getenv("OPENROUTER_API_KEY")
OPENROUTER_BASE_URL = os.getenv("OPENROUTER_BASE_URL",
    "https://openrouter.ai/api/v1")
OPENROUTER_MODEL = os.getenv("OPENROUTER_MODEL", "openai/gpt-4.1-
mini")

client = OpenAI(base_url=OPENROUTER_BASE_URL,
    api_key=OPENROUTER_API_KEY)
app = FastAPI(title="Insurance Helpdesk Bot Backend")

class ClaimRequest(BaseModel):
    claim_note: str

def call_llm(prompt):
    response = client.chat.completions.create(
        model=OPENROUTER_MODEL,
        messages=[
            {"role": "system", "content": "You are an insurance
automation assistant."},
            {"role": "user", "content": prompt}
        ],
        temperature=0.2
    )
    return response.choices[0].message.content

@app.post("/claim-assist")
def claim_assist(request: ClaimRequest):

```

```

prompt = f"""
Analyze the insurance claim note and return:
- Claim Summary
- Claim Type
- Risk Level
- Missing Documents
- Recommended Action

Claim Note:
{request.claim_note}
"""

result = call_llm(prompt)
return {"status": "success", "ai_response": result}

```

Code Element	Explanation	Insurance Relevance
FastAPI()	Creates the API application	Backend for insurance assistant
BaseModel	Defines expected input structure	Prevents missing claim_note
load_dotenv()	Loads environment variables	Keeps API key safe
call_llm()	Reusable LLM call function	Used by many assistants
@app.post("/claim-assist")	Creates POST endpoint	UiPath can call this endpoint
return {"status": ...}	Returns JSON response	Easy for UiPath to consume

How to Run

Save the code as app.py. Run: `uvicorn app:app --reload`. Open Swagger UI at: <http://127.0.0.1:8000/docs>. Click POST /claim-assist, choose Try it out, paste claim_note JSON and Execute.

Hands-on Lab 1: Test FastAPI Claim Assistance

- Create app.py using the given code.
- Run `uvicorn app:app --reload`.

- Open Swagger UI.
- Submit the sample health claim note.
- Observe the response structure.
- Discuss how UiPath can call this endpoint using HTTP Request activity.

```
{
  "claim_note": "Customer Rajesh Sharma submitted a health
insurance claim for dengue treatment at City Care Hospital. Claim
amount is INR 85,000. Policy number is POL1001. Hospital bill and
discharge summary are attached."
}
```

Mini Assignment 1

Enhance /claim-assist so that it also returns: customer sentiment, urgency level and next queue name. Suggested queues: Document Verification, Risk Review, Customer Support, Claim Processing.

Part C: LangChain Overview for Insurance Knowledge Assistants

3. What is LangChain?

- LangChain is a framework for building applications around LLMs.
- It helps organize prompts, models, chains, tools, output parsers and retrieval workflows.
- Instead of writing one large prompt repeatedly, LangChain helps create reusable building blocks.
- For insurance teams, LangChain can support policy FAQ assistants, claim support bots and structured response generation.

Need	Without LangChain	With LangChain
Reusable prompts	Copy-paste prompts in many files	PromptTemplate centralizes prompts
Structured output	Manual parsing	Output parsers help structure response
Multiple steps	Nested Python functions	Chains create clean flow
Business tools	Custom code scattered	Tool functions can be reused

Assistant design	Hard to maintain	Components are easier to manage
------------------	------------------	---------------------------------

Insurance Use Cases for LangChain

Policy FAQ assistant, policy summary generator, claim status assistant, customer support response generator, missing document checklist, underwriting note generator, escalation summary and compliance-friendly communication assistant.

Part D: LangChain Setup and First Prompt

4. LangChain Setup with OpenAI-Compatible API

- We will use LangChain prompt templates and the OpenAI-compatible client pattern.
- If a LangChain ChatOpenAI wrapper supports your provider, use it directly.
- If provider compatibility is limited, create simple Python wrapper functions and still use LangChain templates and parsers.
- This keeps the training practical and avoids dependency confusion.

```
import os
from dotenv import load_dotenv
from openai import OpenAI
from langchain_core.prompts import PromptTemplate

load_dotenv()
client = OpenAI(
    base_url=os.getenv("OPENROUTER_BASE_URL",
"https://openrouter.ai/api/v1"),
    api_key=os.getenv("OPENROUTER_API_KEY")
)
MODEL = os.getenv("OPENROUTER_MODEL", "openai/gpt-4.1-mini")

def call_llm(prompt):
    response = client.chat.completions.create(
        model=MODEL,
```

```

    messages=[
        {"role": "system", "content": "You are an insurance
knowledge assistant."},
        {"role": "user", "content": prompt}
    ],
    temperature=0.2
)
return response.choices[0].message.content

```

Part E: Prompt Templates and Chains

5. Prompt Templates

- A prompt template is a reusable prompt with placeholders.
- It is useful when the same structure is used with different policy texts or customer questions.
- Templates reduce copy-paste mistakes and improve consistency.
- For enterprise training, prompt templates are the first step toward governance.

```

policy_faq_template = PromptTemplate.from_template("""
You are an insurance policy FAQ assistant.

```

Rules:

- Answer only from the provided policy text.
- Do not invent policy benefits.
- Use simple customer-friendly language.
- If information is not available, say so clearly.

Policy Text:

```
{policy_text}
```

Customer Question:

```
{question}
```

```
""")
```

```
policy_text = """
Health Secure Plus covers hospitalization up to INR 5,00,000.
Room rent is covered up to INR 5,000 per day.
Pre-hospitalization is covered for 30 days.
Post-hospitalization is covered for 60 days.
Cosmetic treatment is excluded.
"""
```

```
question = "Is room rent covered?"
prompt = policy_faq_template.format(policy_text=policy_text,
question=question)
answer = call_llm(prompt)
print(answer)
```

Hands-on Lab 2: Build Policy FAQ Assistant

Participants create a reusable prompt template. They test at least five policy questions: coverage limit, room rent, pre-hospitalization, cosmetic treatment and dental treatment. The assistant must not invent answers.

Practice Question	Expected Behaviour
What is the maximum hospitalization coverage?	Answer from policy text
Is room rent covered?	Mention INR 5,000 per day
Is cosmetic treatment covered?	Mention exclusion
Is dental treatment covered?	Say information not available
How many days of post-hospitalization are covered?	Mention 60 days

Mini Assignment 2

Add a field named `response_type`. If `response_type` is `customer`, return simple language. If `response_type` is `internal`, return operational language for the claim team.

Part F: Reusable Chains

6. What is a Chain?

- A chain is a sequence of steps that transforms input into output.
- In simple training labs, a chain can be a function that formats a prompt and calls the LLM.
- Later, chains can connect prompts, tools, retrievers, parsers and business rules.
- For insurance, chains improve reuse across policy summary, FAQ and support workflows.

```
def policy_faq_chain(policy_text, question):
    prompt = policy_faq_template.format(
        policy_text=policy_text,
        question=question
    )
    return call_llm(prompt)

print(policy_faq_chain(policy_text, "Is cosmetic treatment
covered?"))
```

Trainer Explanation

Explain this function as a reusable UiPath workflow equivalent. Input comes in, prompt is prepared, LLM is called, answer comes out. The same chain can be called from FastAPI, Jupyter Notebook or UiPath.

Part G: Output Parsers and Structured Responses

7. Why Structured Output Matters

- UiPath works best with structured data.
- Free-text LLM output is hard to route, validate and audit.
- Structured responses can include answer, confidence, source, missing information and escalation flag.
- Output parsers help define the expected structure and reduce downstream parsing issues.

```

from pydantic import BaseModel, Field
from typing import List

class PolicyAnswer(BaseModel):
    answer: str = Field(description="Customer-friendly answer")
    confidence: str = Field(description="High, Medium or Low")
    source_reference: str = Field(description="Relevant line from
policy text")
    escalation_required: bool = Field(description="Whether human
review is required")

structured_policy_prompt = PromptTemplate.from_template("""
You are an insurance policy assistant.
Return the answer in JSON format with these fields:
answer, confidence, source_reference, escalation_required.

Policy Text:
{policy_text}

Question:
{question}
""")

import json

def structured_policy_answer(policy_text, question):
    prompt = structured_policy_prompt.format(
        policy_text=policy_text,
        question=question
    )
    raw = call_llm(prompt)
    return raw

```

Later: parse JSON safely before sending to UiPath.

Hands-on Lab 3: Use Structured Output

Participants create a structured response for policy FAQ. Required fields: answer, confidence, source_reference and escalation_required. Then they discuss which fields UiPath should use for routing.

Mini Assignment 3

Add two fields to the structured response: next_action and ui_path_queue. Example queues: Customer Response, Policy Review, Manual Escalation.

Part H: Customer Support Assistant

8. Customer Support Assistant Design

- Customer support assistants draft professional responses.
- They must not promise claim approval or policy benefits without evidence.
- They should identify missing information and suggest next action.
- Tone should be clear, polite and compliance-friendly.

```
support_template = PromptTemplate.from_template("""
You are a professional insurance customer support assistant.
```

Rules:

- Do not promise claim approval.
- Do not use emotional or exaggerated language.
- Mention current known status.
- Ask for missing documents if required.
- Keep the response concise and professional.

Customer Message:

```
{customer_message}
```

Claim Context:

```
{claim_context}
```

```
""")
```

```
def customer_support_chain(customer_message, claim_context):
    prompt = support_template.format(
        customer_message=customer_message,
        claim_context=claim_context
    )
    return call_llm(prompt)
```

```
customer_message = """
```

```
Dear Team, I submitted my hospital claim last week.
Please confirm whether it is approved.
```

```
"""
```

```
claim_context = """
```

```
Claim is pending document verification.
Hospital bill and discharge summary are received.
Doctor prescription is pending.
"""
```

```
print(customer_support_chain(customer_message, claim_context))
```

Hands-on Lab 4: Build Customer Support Assistant

Participants generate a professional email response and then create a short SMS version. They compare tone, length and risk of over-commitment.

Part I: Claim Status Assistant

9. Claim Status Assistant

- Claim status assistants translate backend status into clear business messages.
- They can explain whether a claim is pending, under review, missing documents or escalated.
- They are useful for both internal teams and customer communication.
- The assistant should use only provided claim status data.

```
claim_status_template = PromptTemplate.from_template("""
You are an insurance claim status assistant.
```

```
Create a status response from the provided claim data.
```

```
Rules:
```

- Use only the provided data.
- Do not invent approval status.
- Mention missing documents clearly.
- Provide next action.

```
Claim Data:
```

```
{claim_data}
```

```
Audience:
```

```
{audience}
```

```
""")
```

```
def claim_status_chain(claim_data, audience="customer"):
    prompt = claim_status_template.format(
        claim_data=claim_data,
        audience=audience
    )
    return call_llm(prompt)
```

Mini Assignment 4

Create two outputs from the same claim data: one for customer and one for internal operations team. Discuss how the language should change.

Part J: Tools and Simple Agents

10. Tools and Simple Agents

- A tool is a function that performs a specific task.

- Examples: policy lookup, claim status lookup, premium status check and document checklist generation.
- A simple agent decides which tool to use based on user question.
- For beginner training, we simulate agent behaviour with Python logic before using advanced agent frameworks.

```
policy_database = {
    "POL1001": {
        "policy_type": "Health Secure Plus",
        "coverage": "Hospitalization up to INR 5,00,000",
        "status": "Active",
        "premium_status": "Paid"
    },
    "POL2002": {
        "policy_type": "Vehicle Protect",
        "coverage": "Own damage and third-party coverage",
        "status": "Active",
        "premium_status": "Pending"
    }
}

def lookup_policy(policy_number):
    return policy_database.get(policy_number, "Policy not found")

print(lookup_policy("POL1001"))

def simple_insurance_agent(user_question):
    if "policy" in user_question.lower():
        return lookup_policy("POL1001")
    if "claim" in user_question.lower():
        return "Claim is pending document verification."
    return "Please provide policy number or claim number."
```

```
print(simple_insurance_agent("What is the status of policy
POL1001?"))
```

Hands-on Lab 5: Create Simple Insurance Agent

Participants create two tools: `lookup_policy()` and `lookup_claim_status()`. Then they build a simple routing function that selects a tool based on customer question.

Humour for Trainer

Tell participants: “An agent without tools is like an intern with confidence but no system access.” Then explain why tools ground the assistant in business data.

Part K: Reusable Policy Summary Chain

11. Policy Summary Generator

- Policy documents can be lengthy and difficult for customers or operations teams.
- A policy summary chain converts policy text into role-specific summaries.
- Customer summary should be simple and non-legalistic.
- Operations summary should focus on limits, exclusions, required checks and escalation points.

```
policy_summary_template = PromptTemplate.from_template("""
You are an insurance policy summarization assistant.
```

```
Create a policy summary for the selected audience.
```

```
Audience: {audience}
```

```
Include:
```

- Coverage limit
- Key benefits
- Exclusions
- Required checks
- Escalation points

```
Policy Text:
{policy_text}
""")
```

```
def policy_summary_chain(policy_text, audience="customer"):
    prompt = policy_summary_template.format(
        policy_text=policy_text,
        audience=audience
    )
    return call_llm(prompt)
```

Hands-on Lab 6: Policy Summary Generation

Participants generate three versions of the same policy summary: customer-friendly summary, claim processor summary and operations manager summary.

Mini Assignment 5

Add structured output to policy_summary_chain with fields: summary, coverage_limit, exclusions, missing_information and recommended_next_action.

Part L: FastAPI + LangChain Integration

12. Expose LangChain Assistant Through FastAPI

- Once chains are ready, expose them through FastAPI endpoints.
- UiPath can call these endpoints using HTTP Request activity.
- Swagger UI helps test endpoints before UiPath integration.
- Each assistant can become a separate endpoint.

```
@app.post("/policy-faq")
def policy_faq(request: PolicyFAQRequest):
    answer = policy_faq_chain(
        policy_text=request.policy_text,
        question=request.question
    )
    return {
```

```

    "status": "success",
    "answer": answer
  }

```

Suggested API Endpoints

POST /claim-assist, POST /policy-faq, POST /customer-support, POST /claim-status, POST /policy-summary. Each endpoint should accept clean JSON input and return structured JSON output.

Final Assignment: Insurance Policy Assistant using LangChain

Assignment Outcome

- Build an Insurance Policy Assistant using LangChain prompt templates, reusable chains and structured outputs.
- Expose at least one assistant through FastAPI.
- Test using Swagger UI.
- Prepare output that can be consumed by UiPath.

Required Feature	Description	Minimum Expectation
Policy FAQ Assistant	Answers customer policy questions	Uses policy text and question
Customer Support Assistant	Drafts professional response	Avoids false claim promises
Claim Status Assistant	Explains claim status	Mentions missing documents
Output Parser / Structured Response	Returns structured output	Fields useful for UiPath
Reusable Policy Summary Chain	Summarizes policy text	Customer and internal versions
FastAPI Endpoint	Exposes assistant as API	Tested through Swagger UI

Final Deliverables

Participants submit app.py, notebook or Python script, sample policy text, sample questions, screenshots of Swagger UI testing and sample JSON response for UiPath.

Assessment Area	Weightage
Working FastAPI endpoint	20%
Correct LangChain prompt templates	20%
Insurance-domain relevance	20%
Structured response quality	20%
Code readability and reuse	10%
Discussion and explanation	10%

Discussion and Review

Trainer Discussion Questions

- Why should policy FAQ assistants answer only from provided policy text?
- Why is Swagger UI useful before UiPath integration?
- What can go wrong if LLM output is free text only?
- Where should human review be mandatory in insurance workflows?
- How can LangChain improve governance compared to scattered prompts?
- Which assistant should be built first for an insurance client: FAQ, claim status or customer support?

Day 4 Summary

- FastAPI exposes AI logic as APIs for UiPath and enterprise systems.
- Swagger UI helps test APIs quickly and visually.
- LangChain organizes prompts, chains, tools and structured responses.
- Policy FAQ, support, claim status and policy summary assistants are practical insurance use cases.
- Structured outputs are critical for UiPath consumption and enterprise auditability.

End of Day 4 | Next: Agentic AI and LangGraph Insurance Workflows